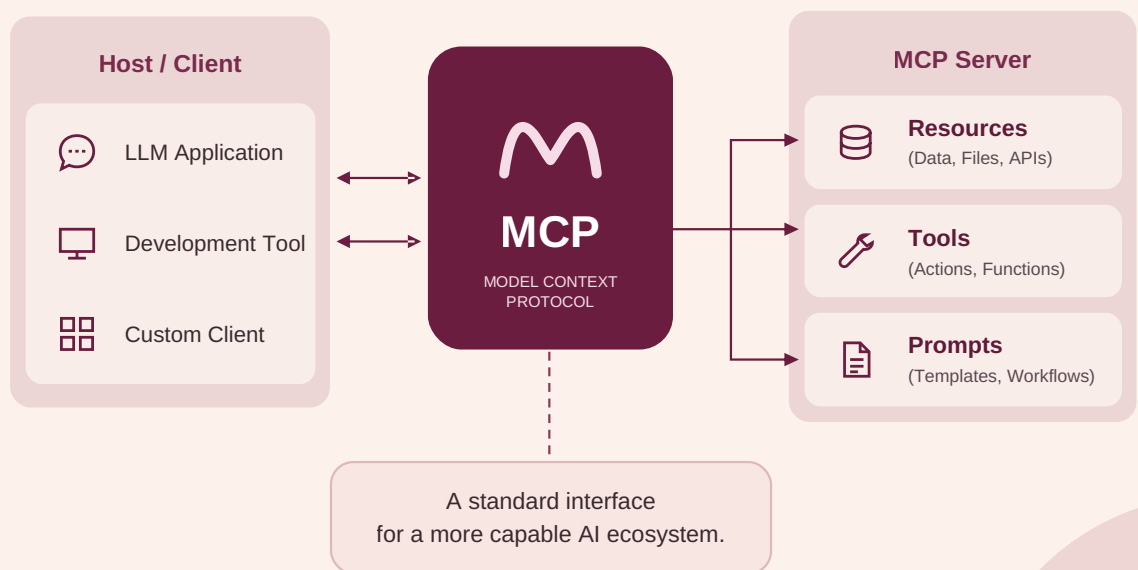




CHAPTER 07

Building Remote MCP Servers

Serve MCP over Streamable HTTP, expose it through a tunnel, make it async, and see why remote servers need authentication.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

Building Remote MCP Servers

A **local** MCP server lives and dies on the machine that runs the host. A **remote** MCP server runs somewhere else and is reached over a network, so many clients can use the same server. This chapter turns the Expense Tracker from the previous chapter into a remote server: it changes the transport, tests the endpoint, exposes it to the internet through a *Cloudflare tunnel*, connects Claude Desktop to it, makes it asynchronous, surveys other deployment options, and ends with the flaw that blocks real multi-user use: **no authentication and no per-user data**.

Streamable HTTP

MCPServer.run()

MCP Inspector

cloudflared tunnel

Custom connectors

mcp-remote bridge

async +aiosqlite

Authentication gap

IMPORTANT

SDK baseline. All code in this chapter targets the **MCP Python SDK v2** (current stable line, installed as `mcp[cli]`). The server class is `MCPServer` imported from `mcp.server`. In v2, *transport options belong to* `run()`, never to the constructor. Tutorials that use a different server class name and import path were written for the v1 line and must not be copied into a v2 project; the SDK Migration Guide lists every change.

1. Local vs Remote MCP Servers

A local server is started by the host as a child process and talks over **stdio**. A remote server is an ordinary network service that speaks MCP over **Streamable HTTP**. The tools, resources and prompts are written the same way in both cases; only how the bytes travel changes.

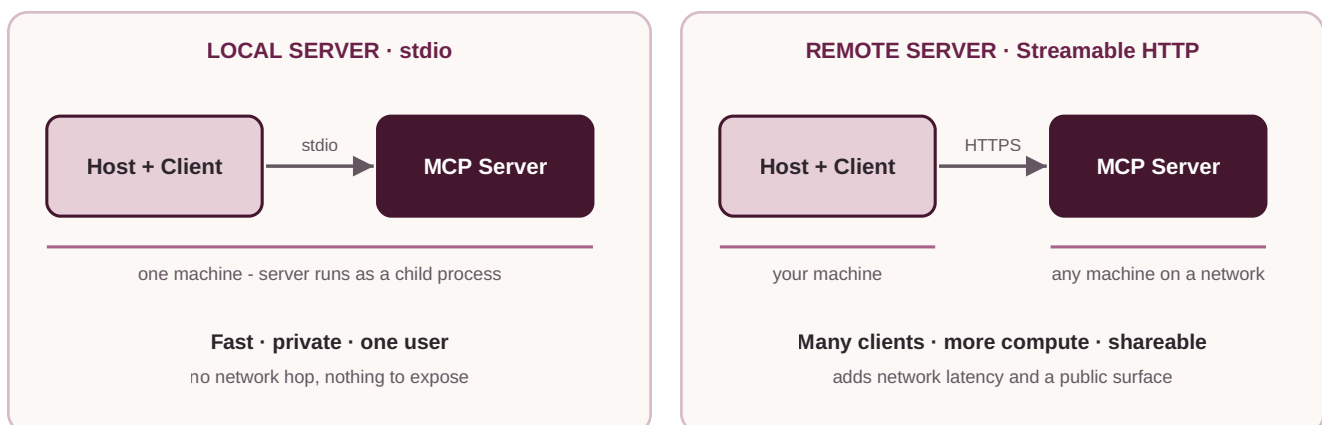


Figure 7.1 — Same server logic, two transports: stdio on one machine versus HTTPS across a network.

Aspect	Local server (stdio)	Remote server (Streamable HTTP)
Where it runs	Same machine as host and client	Any machine reachable over a network
Who can use it	One user, on that machine	Many clients at once
Compute	Limited to the user's machine	Can sit on a far more powerful machine
Speed	Fastest: no network hop	Slower: every call crosses the network
Setup for the user	Config entry that launches a command	Paste a URL (or bridge to it)
Typical owner	Individual developer tools	Companies exposing a service to many users

REAL WORLD INSIGHT

Most enterprise MCP servers will be remote: a company runs one server next to its data and every employee's client connects to it. That is why remote servers are worth learning even though a local server is faster for personal use.

🔄 Quick Revision

- Local = stdio subprocess on your machine; remote = HTTP service on another machine.
- Remote wins on multi-client access and compute; local wins on speed and privacy.
- Tool, resource and prompt code is written identically for both; the transport is the difference.

2. What Changes When a Server Goes Remote

Almost nothing in the server body changes. The only change that matters is the last lines of the file, where `mcp.run()` is told which transport to use. With no argument, `run()` serves over stdio. Naming `streamable-http` makes the SDK build a Starlette application and serve it with uvicorn.

```
# Running the server
if __name__ == "__main__":
    # The modern SDK handles network listening directly inside mcp.run()!
    # 'host="0.0.0.0"' allows connections from the public web interface.
    mcp.run(
        transport="streamable-http",
        host="0.0.0.0",
        port=10000
    )
```

Argument to run()	Meaning	Default
transport	stdio , streamable-http or legacy sse	stdio
host	Interface to listen on; 0.0.0.0 accepts connections from any address	127.0.0.1
port	TCP port (the source uses 10000)	8000
streamable_http_path	URL path of the MCP endpoint	/mcp
json_response=True	Answer each POST with one JSON body instead of an event stream	off
stateless_http=True	Legacy-client sessions only: fresh transport per request	off

The server below is therefore reachable at `http://127.0.0.1:10000/mcp` on the host machine. The `/mcp` suffix is part of the URL every client must use.

COMMON MISTAKE

Passing `host=` or `port=` to `MCPServer(...)` raises `TypeError: MCPServer.__init__() got an unexpected keyword argument 'port'`. The constructor describes *what the server is* (name, instructions, log level); `run()` describes *how it is served*.

The Host header and DNS-rebinding protection

When a server is built for localhost, the SDK arms *DNS-rebinding protection*: only requests whose `Host` header is `127.0.0.1`, `localhost` or `:::1` are accepted. Passing a non-localhost `host` such as `0.0.0.0` stops that default from arming, so every Host and Origin is accepted. This is why the source works unchanged behind a tunnel, whose requests carry a public hostname. It does *not* allowlist the hostname; it simply removes the check.

For a hardened deployment the SDK expects an explicit allowlist through `transport_security`. Without it, a server built for localhost and placed behind a real hostname answers every request with `421 Misdirected Request / Invalid Host header`, a plain-text response that clients surface only as a generic transport error.

What v2 changed underneath

- **Two protocol eras, one app.** The same HTTP app answers clients on the 2026-07-28 revision and legacy clients on earlier revisions, with no flag to switch.
- **No session handshake for modern clients.** Each request is one self-contained POST with no `Mcp-Session-Id`, so any worker can serve any request.
- **Legacy clients keep sessions.** For 2025-era clients the default is a session held in one process's memory, which is why multi-worker deployments need sticky routing or `stateless_http=True` for them.

🔗 Quick Revision

- Going remote = change the transport in `run()`: `transport="streamable-http"`, plus `host` and `port`.
- The endpoint is `/mcp` by default; clients must include it.
- Transport options go to `run()`, never to `MCPServer(...)`.
- Non-localhost `host` disables the localhost Host check; `transport_security` is the explicit allowlist.
- Modern (2026-07-28) requests are sessionless by construction.

3. Project Setup, Running and Testing

Create a fresh `uv` project, add the SDK and the async SQLite driver, and place `main.py` and `categories.json` in the project folder. The categories file must sit next to `main.py` because the server resolves it from `__file__`.

```
cd %USERPROFILE%\Desktop
mkdir remote-expense-server
cd remote-expense-server
uv init .
uv add "mcp[cli]" aiosqlite
uv run main.py
```

On start the server prints the database path and begins listening on `0.0.0.0:10000`. Keep this terminal open; closing it stops the server.

Testing with the MCP Inspector

The Inspector must connect to the running HTTP server as a client. The helper `uv run mcp dev main.py` is not suitable here: it launches the file itself over `stdio` and never reaches the `run(...)` block. Start the Inspector on its own and point it at the endpoint:

```
npx @modelcontextprotocol/inspector
```

1. Set **Transport Type** to **Streamable HTTP**.
2. Enter the URL `http://localhost:10000/mcp` and press **Connect**.
3. Open **Tools** → **List Tools**: `add_expense`, `list_expenses` and `summarize` appear. Open **Resources** to find `expense:///categories`.
4. Run `add_expense` with a date, amount and category, then `list_expenses` over a range that includes that date.

BEST PRACTICE

Always verify the server in the Inspector before exposing it. If tools list and run correctly locally, any later failure is in the tunnel, the connector or the client, not in the server code.

🔄 Quick Revision

- `uv init .`, then `uv add "mcp[cli]" aisqlite`; run with `uv run main.py`.
- Test with the standalone Inspector using Streamable HTTP and `http://localhost:10000/mcp`.
- `mcp dev` runs a file over stdio, so it cannot test the HTTP transport.
- Keep `categories.json` beside `main.py`.

4. The Expense Tracker Server, Walked Through

The attached `main.py` is the implementation source of truth for this chapter. It exposes three async tools and one resource over a SQLite database.

Setup, paths and database initialisation

```
import os
import json
import random
from mcp.server import MCPServer

import aisqlite
import tempfile
# Use temporary directory which should be writable
TEMP_DIR = tempfile.gettempdir()
DB_PATH = os.path.join(TEMP_DIR, "expenses.db")
CATEGORIES_PATH = os.path.join(os.path.dirname(__file__), "categories.json")

print(f"Database path: {DB_PATH}")
mcp = MCPServer("ExpenseTracker")
```

- **Writable location.** The database lives in the operating system's temporary directory (`tempfile.gettempdir()`) instead of beside the code, because the code folder on a deployed host may be read-only. A read-only database was the failure that forced this design: inserts fail with a *readonly* error.
- **Aiosqlite.** Tools use `aiosqlite`, the asynchronous SQLite driver, so database calls can be awaited.
- **Server name.** `MCPServer("ExpenseTracker")` sets the name clients see; it carries no transport options.

`init_db()` deliberately stays synchronous. It runs once at import, before the server starts serving, and uses plain `sqlite3` to enable `WAL` journaling (readers do not block the writer), create the `expenses` table if absent, and prove write access by inserting and deleting a throwaway row. If that test fails, the process stops at startup with a clear error instead of failing later inside a tool call.

```

def init_db(): # Keep as sync for initialization
    try:
        # Use synchronous sqlite3 just for initialization
        import sqlite3
        with sqlite3.connect(DB_PATH) as c:
            c.execute("PRAGMA journal_mode=WAL")
            c.execute("""
                CREATE TABLE IF NOT EXISTS expenses(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    date TEXT NOT NULL,
                    amount REAL NOT NULL,
                    category TEXT NOT NULL,
                    subcategory TEXT DEFAULT '',
                    note TEXT DEFAULT ''
                )
            """)
            # Test write access
            c.execute("INSERT OR IGNORE INTO expenses(date, amount, category) VALUES
('2000-01-01', 0, 'test')")
            c.execute("DELETE FROM expenses WHERE category = 'test'")
            print("Database initialized successfully with write access")
    except Exception as e:
        print(f"Database initialization error: {e}")
        raise

# Initialize database synchronously at module load
init_db()

```

Async tools

Every tool follows one pattern. Read the three points below, then the code that applies them.

- **async def** makes the tool a coroutine, so the server can run other requests while this one waits.
- **async with aiosqlite.connect(...)** opens the connection without blocking; **await c.execute** and **await c.commit()** yield control during I/O.
- **Parameterised SQL.** Values travel as **?** placeholders, never concatenated into the statement, which prevents SQL injection.
- **Structured results.** **add_expense** returns a dictionary with **status**, the new **id** and a message; its **readonly** branch turns a permissions failure into a readable error instead of a crash.

```

@mcp.tool()
async def add_expense(date, amount, category, subcategory="", note=""):
    '''Add a new expense entry to the database.'''
    try:
        async with aiosqlite.connect(DB_PATH) as c:
            cur = await c.execute(
                "INSERT INTO expenses(date, amount, category, subcategory, note) VALUES
                (?, ?, ?, ?, ?)",
                (date, amount, category, subcategory, note)
            )
            expense_id = cur.lastrowid
            await c.commit()
            return {"status": "success", "id": expense_id, "message": "Expense added
            successfully"}
    except Exception as e:
        if "readonly" in str(e).lower():
            return {"status": "error", "message":
                "Database is in read-only mode. Check file permissions."}
        return {"status": "error", "message": f"Database error: {str(e)}"}

```

`list_expenses` returns rows between two dates (inclusive), newest first, as a list of dictionaries built from `cur.description`. `summarize` assembles its query in steps: the base filters on the date range, an optional category adds `AND category = ?`, then results are grouped by category with a total and a count. Only fixed SQL text is concatenated; all values remain parameters.

```

@mcp.tool()
async def list_expenses(start_date, end_date):
    '''List expense entries within an inclusive date range.'''
    try:
        async with aiosqlite.connect(DB_PATH) as c:
            cur = await c.execute(
                """
                SELECT id, date, amount, category, subcategory, note
                FROM expenses
                WHERE date BETWEEN ? AND ?
                ORDER BY date DESC, id DESC
                """,
                (start_date, end_date)
            )
            cols = [d[0] for d in cur.description]
            return [dict(zip(cols, r)) for r in await cur.fetchall()]
    except Exception as e:
        return {"status": "error", "message": f"Error listing expenses: {str(e)}"}

```

```

@mcp.tool()
async def summarize(start_date, end_date, category=None):
    '''Summarize expenses by category within an inclusive date range.'''
    try:
        async with aiosqlite.connect(DB_PATH) as c:
            query = """
                SELECT category, SUM(amount) AS total_amount, COUNT(*) as count
                FROM expenses
                WHERE date BETWEEN ? AND ?
            """
            params = [start_date, end_date]

            if category:
                query += " AND category = ?"
                params.append(category)

            query += " GROUP BY category ORDER BY total_amount DESC"

            cur = await c.execute(query, params)
            cols = [d[0] for d in cur.description]
            return [dict(zip(cols, r)) for r in await cur.fetchall()]
    except Exception as e:
        return {"status": "error", "message": f"Error summarizing expenses: {str(e)}"}

```

The categories resource

```

@mcp.resource("expense:///categories", mime_type="application/json")
def categories():
    try:
        # Provide default categories if file doesn't exist
        default_categories = {
            "categories": [
                "Food & Dining",
                # ... eight more default categories omitted ...
                "Other"
            ]
        }

    except FileNotFoundError:
        import json
        return json.dumps(default_categories, indent=2)
    except Exception as e:
        return f'{{"error": "Could not load categories: {str(e)}"}}'

```

The URI `expense:///categories` has the scheme `expense`, an empty authority and the path `/categories`; the third slash is required. The resource returns the contents of `categories.json` (a mapping from each category to its

subcategories) so a client can offer valid values to the model. If the file is missing, the code falls back to a built-in default list.

IMPORTANT

The fallback and the file have **different shapes**: the file is a mapping of category to subcategory lists, while the fallback is `{"categories": [...]}`, a flat list. A client written against one shape would misread the other, so ship `categories.json` with the code and keep the two shapes aligned.

BEST PRACTICE

The tool parameters in the source are un-annotated (`date`, `amount`, `category`). Adding type hints such as `date: str`, `amount: float` gives clients a precise input schema and catches malformed calls early. This is a recommended hardening step and is not part of the supplied source.

Quick Revision

- Database path is `tempfile.gettempdir()/expenses.db` : writable on hosts where the code folder is not.
- `init_db()` is sync and runs once at import; it enables WAL and proves write access.
- All three tools are `async` and use `aiosqlite` with `await` on execute, fetch and commit.
- SQL values are always `?` parameters; errors return `{"status": "error", ...}`.
- Resource URI is `expense:///categories` ; the file and the fallback have different shapes.

5. Exposing the Server with a Cloudflare Tunnel

A server on `localhost:10000` is invisible to anything outside your machine. A *tunnel* gives it a public HTTPS address without a cloud deployment: a small program on your machine, **cloudflared**, opens an outbound connection to Cloudflare, and Cloudflare forwards requests arriving at a public hostname back through that connection to your local port. This is a good way to learn remote servers because the server, the data and the logs stay on your own computer.

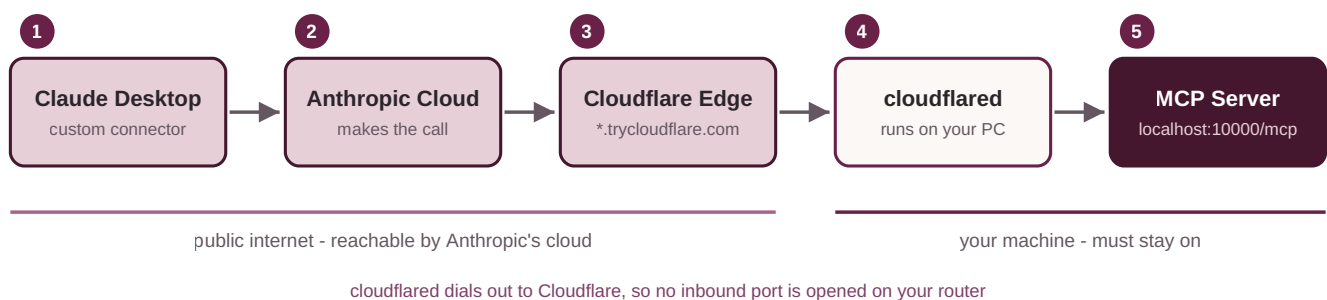


Figure 7.2 — A Claude custom connector reaches a local server through Anthropic's cloud, a Cloudflare hostname and an outbound cloudflared connection.

```
winget install --id Cloudflare.cloudflared
where cloudflared
cloudflared tunnel --url http://localhost:10000
```

Run the tunnel in a *second* terminal while the server runs in the first. cloudflared prints a random address of the form `https://<random-words>.trycloudflare.com`. Your MCP URL is that address with `/mcp` appended. Test it first in the Inspector (Streamable HTTP, the public URL) before involving Claude.

Quick tunnel (used here)		Named tunnel
Account needed	No	Cloudflare account and a domain
URL	Random, changes on every restart	Stable hostname you control
Access control	None built in	Can be fronted by access policies
Intended use	Development and demos	Longer-lived or production-style access
Documented limits	No uptime guarantee; 200 concurrent in-flight requests; no Server-Sent Events	Not subject to the quick-tunnel limits

IMPORTANT

Why the URL must be public. A custom connector added in Claude is called from *Anthropic's cloud*, not from your computer. A `localhost` address therefore cannot work for that route; a publicly reachable HTTPS URL, here the tunnel, is required.

COMMON MISTAKE

Anyone who has the URL can call your tools. A quick tunnel adds no authentication, and this server has none either. Treat the address like a password, stop the tunnel when you are finished, and never point a tunnel at a database holding real data. If event streams stall behind a quick tunnel, `json_response=True` in `run()` avoids streaming; the cost is that tools cannot call back into the client mid-request, which this server never does.

🔄 Quick Revision

- A tunnel publishes a local port at a public HTTPS hostname via an outbound connection; no router changes.
- Command: `cloudflared tunnel --url http://localhost:10000` ; MCP URL = printed address + `/mcp`.
- Quick-tunnel URLs are random and change on restart; they are for development only.
- Server, tunnel and your computer must all stay on for the remote server to work.
- Custom connectors are called from Anthropic's cloud, so they need a public URL.

6. Connecting Claude Desktop to a Remote Server

There are two ways to attach a remote server to Claude Desktop, and they differ in *who makes the call*.

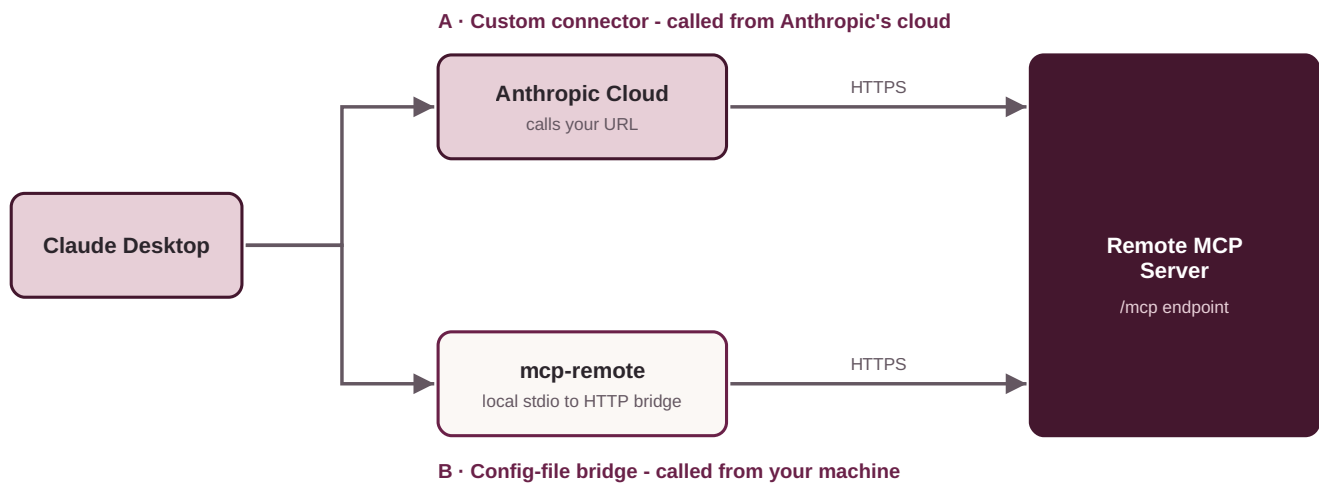


Figure 7.3 — Route A is called from Anthropic's cloud; Route B is a local bridge process that calls the server from your machine.

Route A: custom connector (no file editing)

1. In Claude, open **Customize** → **Connectors**, choose + **Add**, then **Add custom connector**.
2. Enter a name and the full URL, for example `https://<your-tunnel>.trycloudflare.com/mcp`.
3. Add the connector. Some versions also offer *Request headers* for fixed credentials such as an API key; the server in this chapter needs none.
4. Open a chat, ask for an expense to be added, and approve the tool call when prompted. The tools `add_expense`, `list_expenses` and `summarize` appear under the connector.

Custom connectors are available on Free (limited to one), Pro, Max, Team and Enterprise plans. Team and Enterprise organisations require an owner to add the connector before members connect. Always check the current Claude help pages, since limits change.

Route B: a bridge in `claude_desktop_config.json`

Claude Desktop launches local programs from its config file. The `mcp-remote` package is a local program that speaks stdio to Claude on one side and HTTP to your remote server on the other, so Claude believes it is talking to a local server. The file follows the usual schema: a top-level `mcpServers` object with `command` and `args` per entry. Open it from **Settings** → **Developer** → **Edit Config** (on Windows it lives under `%APPDATA%\Claude`).

```
{
  "mcpServers": {
    "expense-tracker-remote": {
      "command": "npx",
      "args": [
        "-y",
        "mcp-remote",
        "https://<your-tunnel>.trycloudflare.com/mcp",
        "--transport",
        "http-only"
      ]
    }
  }
}
```

Restart Claude Desktop completely after saving. If it cannot find `npx`, replace `"npx"` with the full path printed by `where npx`, the same way the other entries in the file use full executable paths.

	A: Custom connector	B: mcp-remote bridge
Configured in	Claude settings UI	claude_desktop_config.json
Request originates	Anthropic's cloud	Your own machine
Server must be reachable from	The public internet	Your machine (a LAN or localhost URL also works)
Extra moving part	None	A local Node process per server
Latency	Direct	One extra hop through the bridge
Use when	Normal case; sharing with teammates	Policy blocks connectors, or you want a config-file entry

BEST PRACTICE

When a quick-tunnel URL changes, the connector or bridge entry still points to the old address and silently fails. Update the URL in the connector (Route A) or the `args` list (Route B) every time you restart the tunnel.

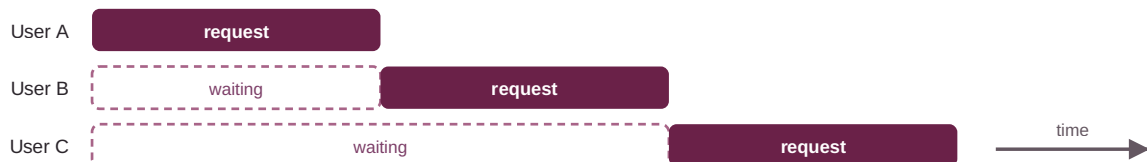
Quick Revision

- Route A: Customize → Connectors → Add custom connector, URL ends in `/mcp` ; called from Anthropic's cloud.
- Route B: `npx -y mcp-remote <url> --transport http-only` in `mcpServers` ; called from your machine.
- The bridge turns a remote server into something that looks local to Claude Desktop.
- Restart Claude Desktop after editing the config file.
- A changed tunnel URL breaks both routes until updated.

7. Concurrency: From Blocking to Async

A server shared by many users must not make them queue. If a tool performs its database work synchronously, the single worker is *blocked* for the whole call: while one user's `add_expense` waits on the database, a second user's request cannot even start. The fix is Python's `async` / `await`: a waiting tool hands control back to the event loop, which serves other requests until the data is ready.

BLOCKING (synchronous tools)



ASYNC (async tools + aiosqlite)



Figure 7.4 — Blocking tools serve users one after another; async tools overlap their waiting time.

Concern	Blocking version	Async version in main.py
Tool definition	<code>def add_expense(...)</code>	<code>async def add_expense(...)</code>
Database driver	<code>sqlite3</code>	<code>aiosqlite</code>
Connection	<code>with sqlite3.connect(...)</code>	<code>async with aiosqlite.connect(...)</code>
Query / fetch / commit	Called directly	<code>await c.execute</code> , <code>await cur.fetchall()</code> , <code>await c.commit()</code>
Startup initialisation	Sync	Still sync, once, before serving

IMPORTANT

Async gives **concurrency, not parallelism**: one event loop interleaves requests that are *waiting*. A single blocking call inside an `async def`, such as `time.sleep`, a sync database call or heavy CPU work, still freezes every other request on that worker.

Concurrency also interacts with SQLite: `WAL` mode lets readers continue while a write happens, but SQLite still allows one writer at a time. That is acceptable for this project; a heavily written, multi-user service would move to a client-server database.

🔄 Quick Revision

- Blocking tools force users into a queue; async tools let waiting overlap.
- Conversion: `def` → `async def`, `sqlite3` → `aiosqlite`, add `await` to every I/O call.
- One blocking call inside an async tool stalls the whole worker.
- WAL helps readers; SQLite still has a single writer.

8. Beyond Tunnels: Other Ways to Deploy

A tunnel is a learning tool. To keep a server reachable without your computer, run the same code on a machine that stays on. The code is unchanged; what differs is where it runs, what it costs and how long the data survives. Pricing and free tiers change often, so treat the table as a map of options and confirm current terms before committing.

Option	Cost model	Fit and caveats
Cloudflare quick tunnel	Free	Your own PC; random URL; development only
Cloudflare named tunnel	Free tunnel; domain required	Stable hostname for a PC or home server; supports access policies
Other tunnel tools (ngrok, Tailscale Funnel)	Free and paid tiers	Same idea with different limits; check current terms
Container platforms (Render, Railway, Fly.io)	Free tiers shrinking; paid usage or monthly	Git-based deploys; free instances may sleep when idle, which suits MCP badly; Fly.io no longer offers new free plans
Serverless / edge (Cloudflare Workers, Vercel)	Free tier plus usage	Best for TypeScript/JavaScript servers; a Python ASGI server may not fit directly
Major clouds (Cloud Run, Azure Container Apps, AWS)	Pay per use, some free quota	Most control and scaling; most configuration
VPS	Fixed monthly fee	Full control; you run uvicorn, proxy and TLS yourself

Production checklist for any host

- **Allowlist the hostname.** Build the ASGI app yourself and pass `transport_security`; otherwise a localhost-built app returns 421 on a real hostname.
- **Trust the proxy.** Behind a TLS-terminating proxy, run uvicorn with `--proxy-headers --forwarded-allow-ips` or redirects point at `http://` and clients refuse them.
- **Persist data.** The source stores SQLite in the temp directory, which container platforms usually wipe on redeploy; use a mounted volume or a hosted database.
- **Secrets in environment variables**, never in code or in Git.
- **Add authorization** (Section 9) before sharing the URL beyond yourself.

The SDK pattern for a real hostname (taken from the SDK documentation, not from `main.py`) builds the app explicitly and is served by any ASGI host:

```
from mcp.server.transport_security import TransportSecuritySettings

security = TransportSecuritySettings(
    allowed_hosts=["mcp.example.com", "mcp.example.com:*"],
    allowed_origins=["https://app.example.com"],
)
app = mcp.streamable_http_app(transport_security=security)

# served with, for example:
# uvicorn main:app --workers 4 --proxy-headers --forwarded-allow-ips=<proxy address>
```

REAL WORLD INSIGHT

Scaling is mostly the platform's job. Modern-protocol requests need no sticky routing, but a tool that asks the user a follow-up question across two requests carries a sealed `request_state`; with several workers, every instance must share the same `RequestStateSecurity` keys *and* the same server name. The Expense Tracker has no such tools, so a single worker is fine.

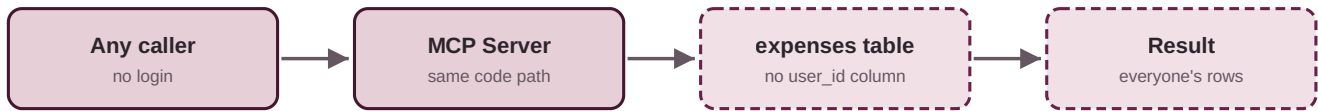
🔄 Quick Revision

- Tunnels are for learning; a real deployment moves the same code to an always-on host.
- Free tiers change: some sleep when idle, some no longer exist. Verify before choosing.
- Go-live gate: `transport_security` allowlist, proxy headers, persistent storage, secrets, auth.
- `run()` starts one process; for more workers, hand `streamable_http_app()` to uvicorn or gunicorn.

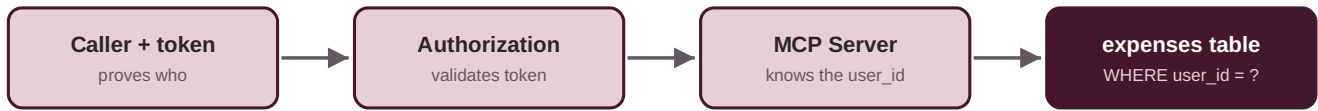
9. The Remaining Flaw: Shared Data and Authentication

The server now works remotely and concurrently, but it is not safe to share. The `expenses` table has **no** `user_id` **column**, and the server has no way to tell callers apart. Everyone who connects reads and writes the same rows: if ten people use the server, each can list, summarise and add to everybody else's expenses.

TODAY - no identity anywhere in the flow



NEEDED - identity travels with every request



A chat window is not authentication: the server must learn the caller's identity from the request itself.

Figure 7.5 — Without identity every caller sees every row; a fix needs authentication first, then a `user_id` filter.

Adding a `user_id` column is the easy half. The hard half is *knowing* which user is calling. Claude chats with the server directly, with no login in between, so a caller who claims to be user 123 cannot be trusted. The identity must come from **authorization** on the connection itself (a validated token), after which each tool writes and filters by that identity.

BEST PRACTICE

A fixed credential in the connector's request headers can keep strangers out of the endpoint, but it identifies the *connection*, not individual users. It is a stopgap for a private server, not a multi-user design. The SDK provides OAuth-based authorization for remote servers; it is the next step.

INTERVIEW TIP

“What is wrong with a remote MCP server that has no auth?” Anyone with the URL can invoke its tools, and a shared database cannot separate users. Remote exposure requires authentication for access control and an identity for per-user data.

🔄 Quick Revision

- No `user_id` + no authentication = every caller shares and sees everything.
- A user ID claimed in chat is not trustworthy; identity must come from validated authorization.
- Fix order: authenticate the connection, then store and filter by `user_id`.
- A fixed header credential protects the endpoint but does not identify users.

10. Troubleshooting Remote Servers

Symptom	Likely cause	Fix
Inspector cannot connect	URL missing <code>/mcp</code> , wrong port, server not running	Use <code>http://localhost:10000/mcp</code> ; confirm the server terminal is running
<code>TypeError ... unexpected keyword argument 'port'</code>	Transport option passed to the constructor	Move <code>host</code> and <code>port</code> into <code>run()</code>
421 / Invalid Host header after deploying	Localhost-built app behind a real hostname	Pass <code>transport_security</code> or a non-localhost host
Tools worked yesterday, fail today	Quick-tunnel URL changed	Copy the new URL into the connector or config
“Database is in read-only mode”	Database path not writable	Keep the DB in <code>tempfile.gettempdir()</code> as the source does
Data disappears after redeploy	Ephemeral disk on the host	Use a persistent volume or hosted database
Bridge entry shows an error in Claude Desktop	<code>npx</code> not found or app not restarted	Use the full path from where <code>npx</code> ; restart Claude Desktop
Responses hang behind a quick tunnel	Quick tunnels do not support event streams	Try <code>json_response=True</code> in <code>run()</code>

🔄 Quick Revision

- Check in this order: server running → Inspector locally → tunnel URL → connector or bridge.
- 421 on a real hostname means the Host allowlist, and the reason appears only in the server log.

Key Takeaways

- **Remote = a different transport.** Tools stay identical; `run(transport="streamable-http", host=..., port=...)` exposes `/mcp`.
- **Test locally first** with the standalone MCP Inspector over Streamable HTTP.
- **A tunnel makes a local server public** through an outbound connection; quick tunnels are temporary and unauthenticated.
- **Claude connects two ways:** a custom connector (called from Anthropic's cloud) or an `mcp-remote` bridge in the config file (called from your machine).
- **Async tools stop users from queueing;** never block the event loop inside them.
- **Going live needs more:** Host allowlist, proxy headers, persistent storage, secrets and authorization.
- **Not safe for many users yet:** no authentication and no `user_id` means everyone shares one dataset.

Revision Sheet

Six cards covering the mechanics worth memorising before an interview or a build.

Transport switch

```
mcp.run() = stdio.  
mcp.run(transport="streamable-http",  
host="0.0.0.0", port=10000) = HTTP at /  
mcp.
```

Constructor vs run()

`MCPServer(name, log_level, debug)` describes the server. Transport options live on `run()`.

Tunnel in one line

```
cloudflared tunnel --url http://  
localhost:10000 → add /mcp to the printed  
HTTPS address.
```

Two Claude routes

Connector: Anthropic's cloud calls your URL. Bridge:
`npx -y mcp-remote <url> --transport http-only` runs locally.

Sync → async

```
async def, aiosqlite, async with, await  
on execute / fetch / commit.
```

Go-live gate

421 = Host allowlist. Proxy headers for TLS proxies.
Persistent DB. Secrets in env. Add authorization.

Interview Questions

- Q1.** What changes in code when you convert a stdio MCP server into a remote one? What stays the same?
- Q2.** Why does `mcp dev server.py` not test a Streamable HTTP server, and how do you test one?
- Q3.** A server built for localhost returns `421 Invalid Host header` after deployment. Why, and what are two fixes?
- Q4.** Why must a custom connector URL be publicly reachable, while an `mcp-remote` bridge can target a local address?
- Q5.** Explain what a tunnel does, and why `cloudflared` needs no inbound port. What are the risks of a quick tunnel?
- Q6.** Why does an async tool let other users proceed, and how can an `async def` tool still block everyone?
- Q7.** Why is the Expense Tracker unsafe to share, and what must exist before a `user_id` column helps?
- Q8.** Why does modern-protocol traffic need no sticky sessions, and what still needs shared configuration across workers?

Further Reading

- [MCP Python SDK: Running your server](#) (transports and `run()` options)
- [MCP Python SDK: Deploy & scale](#) (Host allowlist, proxies, workers, request state)
- [MCP Python SDK: Authorization](#) (the next step after this chapter)
- [MCP Python SDK: Installation](#) and [Migration Guide](#)
- [Claude Help: Get started with custom connectors using remote MCP](#)
- [Claude Help: Building custom connectors via remote MCP servers](#)